Robust Coreness Estimation via H-index under Edge LDP

ZHEYUAN DAI, Shanghai Jiao Tong University, China
XULIANG ZHU*, Antai College of Economics and Management, Shanghai Jiao Tong University, China and Data-Driven Management Decision Making Lab, Shanghai Jiao Tong University, China
YIZHANG HE, Global College, Shanghai Jiao Tong University, China
KAI WANG, Antai College of Economics and Management, Shanghai Jiao Tong University, China and Data-Driven Management Decision Making Lab, Shanghai Jiao Tong University, China
FAN ZHANG, Guangzhou University, China
XUEMIN LIN, The Chinese University of Hong Kong, Shenzhen, China

The coreness of a vertex is the largest integer k such that it belongs to a subgraph in which every vertex has at least k neighbors within that subgraph. Core decomposition identifies dense regions, highlights key vertices, and supports visualization and community discovery in networks. However, releasing exact core numbers may leak private connections between users. This calls for studying core decomposition under local differential privacy, where each user perturbs its own connection information before sharing it. While existing methods offer theoretical guarantees, they often suffer from poor accuracy for high-degree vertices and high computational overhead on large scale, real world graphs. The H-index of a vertex is the largest integer h such that at least h of its neighbors have values no smaller than h. Because the H-index naturally captures latent graph structures through local information propagation, it serves as an effective proxy for coreness in decentralized settings.

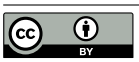
In this paper, we leverage these properties to develop a unified H-index-based framework for core decomposition under LDP, with three progressively refined variants. KcoreH serves as the basic estimator and iteratively updates coreness estimates through distributed H-index computation. Building on KcoreH, KcoreHT introduces coarse-grained vertex freezing to suppress redundant updates and reduce communication. KcoreHA further refines this design through adaptive budget allocation, fine-grained freezing, and online convergence detection, yielding our most accurate variant. Experimental results on 11 real-world graphs demonstrate that our H-index-based approach consistently outperforms the state of the art in estimation accuracy, with KcoreHA achieving an order-of-magnitude improvement on average, while also attaining significantly lower running time and communication costs under rigorous privacy guarantees.

CCS Concepts: • **Security and privacy → Social network security and privacy**; • **Networks → Network privacy and anonymity**.

Additional Key Words and Phrases: Differential Privacy; k-core decomposition

*Xuliang Zhu is the corresponding author.

Authors' Contact Information: Zheyuan Dai, Shanghai Jiao Tong University, China, dzy040811@gmail.com; Xuliang Zhu, Antai College of Economics and Management, Shanghai Jiao Tong University, Shanghai, China and Data-Driven Management Decision Making Lab, Shanghai Jiao Tong University, Shanghai, China, zhu.xl@sjtu.edu.cn; Yizhang He, Global College, Shanghai Jiao Tong University, China, yizhang.he@sjtu.edu.cn; Kai Wang, Antai College of Economics and Management, Shanghai Jiao Tong University, Shanghai, China and Data-Driven Management Decision Making Lab, Shanghai Jiao Tong University, Shanghai, China, cskaelwang@gmail.com; Fan Zhang, Guangzhou University, China, zhangf@gzhu.edu.cn; Xuemin Lin, The Chinese University of Hong Kong, Shenzhen, China, xuemin.lin@gmail.com.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2026 Copyright held by the owner/author(s).
ACM 2836-6573/2026/12-ART388
https://doi.org/10.1145/3856963

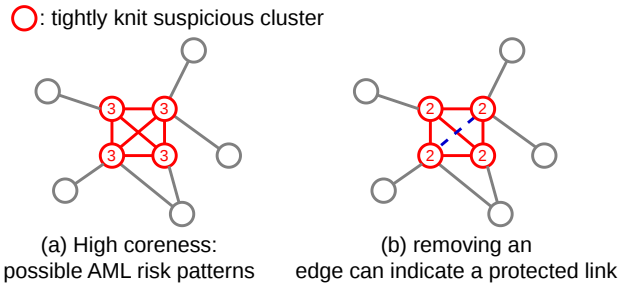


Fig. 1. LEDP enables useful coreness estimation without revealing the true transaction graph.

ACM Reference Format:
Zheyuan Dai, Xuliang Zhu, Yizhang He, Kai Wang, Fan Zhang, and Xuemin Lin. 2026. Robust Coreness Estimation via H-index under Edge LDP. *Proc. ACM Manag. Data* 4, 6 (SIGMOD), Article 388 (December 2026), 26 pages. https://doi.org/10.1145/3856963

1 Introduction

The k-core of a graph is a maximal subgraph in which every vertex has degree at least k [4, 45]. k-core decomposition computes these cores across all k and summarizes a vertex's structural position by its *core number* (coreness), defined as the largest k such that the vertex belongs to the k-core. Core decomposition supports applications such as community discovery [24], influence estimation [8, 35], anomaly detection [48], and structural summarization and visualization [2] across domains including social/communication networks, transaction graphs, and epidemiology contact networks. However, releasing exact coreness information can compromise privacy by revealing sensitive relationships. For example, consider a transaction network in Figure 1, where each edge indicates a money-transfer relationship. If core numbers are released, an adversary may observe discrete changes caused by adding/removing a single edge and infer the existence of a protected transaction link or membership in a sensitive tightly connected group. As illustrated in Figure 1(b), removing a single edge can cause a group with coreness 3 to degrade to coreness 2, thereby revealing the underlying group structure. Banks and regulators therefore aim to identify high coreness users as potential AML risk indicators, while preventing any institution or data curator from accessing the underlying transaction links.

Motivated by these observations, we study the problem of privacy preserving coreness estimation on graphs. Differential privacy (DP) is the standard for rigorous privacy protection [15]. Classical *central* DP assumes a trusted curator with access to the true graph, but modern decentralized deployments often break this assumption and motivate the *local* model [21, 34], where each user randomizes their data before sharing. For graphs, the sensitive units are naturally *edges* (relationships), and local privacy has become a standard approach for sensitive graph analytics, including triangle counting under local privacy [29, 30] and locally private bipartite motif (biclique-style) estimation work [25, 26]. Accordingly, given a graph G, we aim to estimate each vertex's core number under *local edge differential privacy* (LEDP) [10, 30, 43]. Under LEDP, every vertex perturbs its incident edge information before transmission to the data curator, so that the presence or absence of any single edge cannot be reliably inferred from the output.

Existing methods. The existing LEDP coreness estimator, KCoreD [41], adapts iterative peeling to the private setting. Its repeated noisy updates can accumulate estimation error and communication cost, particularly for vertices that remain active for many rounds. This motivates an approach that avoids private peeling decisions and requires fewer interactions.

Challenges. In this paper, we design accurate and efficient algorithms for coreness estimation under LEDP and face the following challenges.

• *Challenge 1:* Coreness depends on recursive global connectivity. Under LEDP, perturbing incident edges obscures this structure, making direct estimation inaccurate and allowing errors to propagate through iterative refinement.

• *Challenge 2:* Accurate coreness estimation typically requires iterative computation. Under LEDP, releasing updates in every iteration can be wasteful, since many vertices stabilize early and continued uploads only accumulate noise and communication cost without improving the final estimates. Table 2 shows estimates converging within the first few rounds, after which further iteration mainly adds noise rather than improving accuracy.

• *Challenge 3:* It is challenging to control communication and privacy-budget usage across rounds while maintaining strong accuracy, since every upload must be locally randomized under LEDP.

Our approaches. To address the above challenges, we build on the neighborhood H-index operator: given neighbor values, it returns the largest integer h such that at least h neighbors have value at least h. Iterating the H-index on true degrees converges to coreness in the noiseless setting [37]. Its key advantage is that repeated local H-index aggregation can recover a quantity that normally depends strongly on the global graph structure, without requiring any party to observe or reconstruct that structure explicitly. Instead, the structural relationships needed for coreness estimation are captured implicitly through the estimates propagated by neighboring vertices.

The operator is also 1-Lipschitz: changing one input neighbor value by one changes the resulting H-index by at most one. Therefore, each local update has sensitivity 1 and can be privatized naturally using symmetric geometric noise, a standard perturbation mechanism for integer-valued statistics under LEDP, while limiting noise amplification across rounds.

Concretely, to address *Challenge 1*, we propose KcoreH, which starts from a noisy degree release and then runs $T-1$ synchronous refinement rounds: each vertex computes the H-index of its neighbors' previous estimates and uploads a new noisy update, with privacy split ε/T per round. Building on this estimator, KcoreHT addresses *Challenge 2* by introducing a transition-based locking rule, so that vertices that stabilize early stop uploading and only the active set continues to communicate. The final refinement, KcoreHA, addresses *Challenge 3* by adaptively allocating the remaining privacy budget across rounds and terminating automatically according to a convergence signal, thereby removing the need to preselect a fixed number of iterations.

Contributions. Here, we outline our main contributions.

• We initiate the study of H-index refinement as a principled primitive for coreness estimation under LEDP, and propose a family of estimators, KcoreH, KcoreHT, and KcoreHA, that refine coreness via locally privatized neighborhood aggregation rather than peeling on a perturbed graph.

• We propose KcoreHT, which augments KcoreH with a transition-based locking rule that suppresses unnecessary uploads after early stabilization. By allowing only an active subset of vertices to continue communicating, KcoreHT reduces communication cost and avoids redundant noise accumulation during iterative refinement under LEDP.

• We propose KcoreHA, which adaptively allocates the remaining privacy budget across rounds and stops automatically once the refinement process has converged. This enables just enough iteration at both the vertex level and the global level, so that stabilized vertices stop uploading early, unresolved vertices continue refining, and the protocol stops once the estimates have converged, thereby reducing communication cost and improving accuracy.

• We provide explicit finite-sample utility guarantees for the proposed estimators under a fixed privacy budget, including high-probability error bounds, which bound the estimation error with probability at least $1-\delta$, and explain how accuracy evolves across rounds and how noise accumulates during iterative refinement.

• We conduct extensive experiments on real graphs (including sensitive transaction-network settings), showing substantial accuracy improvements, reduced communication costs, and lower runtime on most datasets compared with the prior LEDP baseline [41].

2 Related Work

Core decomposition and H-index. The k-core is a classical measure of network cohesion, defined as a maximal subgraph in which every vertex has degree at least k [45]. Efficient exact, dynamic, and streaming algorithms make core decomposition practical on large graphs [4, 36, 38, 44]. Core numbers have been used for influence analysis, epidemic modeling, recommendation, and graph clustering [1, 9, 24, 35, 39, 40, 46, 50]. Most relevant to this study, Lü et al. showed that repeatedly applying the neighborhood H-index to vertex degrees converges to coreness [37]. This result connects coreness with an iterative computation based on local neighborhood values.

Differentially private graph analytics. Differential privacy has been widely studied for graph statistics under both central and local models [15]. In the central model, a trusted curator perturbs outputs such as degree distributions, triangle counts, dense subgraphs, and graph decompositions [5, 12–14, 16, 22, 32, 42]. The local model removes the trusted curator by requiring each user to randomize its own data before transmission. Existing local graph analytics include synthetic graph generation, generic graph statistic estimation, triangle counting, truss decomposition, butterfly counting, and common neighborhood estimation [20, 25–28, 30, 31, 43, 49, 51].

Private coreness estimation. Dhulipala et al. [11] first studied core decomposition under LEDP. Their level based method, built on locally adjustable graph algorithms, gave the first formal privacy and approximation guarantees for this setting, though it remained primarily theoretical and required repeated noisy level updates. Dhulipala et al. [12] later studied the central DP setting, where a trusted curator has full graph access, and obtained near optimal additive error using a generalized sparse vector technique. More recently, Mundra et al. [41] developed KcoreD, the first practical distributed implementation of the level based LEDP framework. KcoreD adds private degree thresholds, bias correction, and repeated noisy level refinement to improve accuracy and scalability, and it also evaluates CoreRR, a baseline that applies randomized response to potential edges before running standard core decomposition. KcoreD is therefore the most recent and directly comparable practical baseline for our setting. Existing LEDP approaches mainly follow graph reconstruction, which suffers from false positive edges and poor scalability on sparse graphs, or noisy level refinement, which requires repeated noisy decisions and communication rounds. We discuss these limitations in detail in Section 4.

3 PRELIMINARIES

In this section, we discuss the fundamental concepts and tools underlying our LEDP based core decomposition framework. We first formalize the notion of coreness estimation under local edge differential privacy, then introduce the H-index operator and its connection to k-core decomposition. Finally, we summarize the privacy primitives and noise mechanisms used in our algorithms. Table 1 summarizes the frequently used notation.

3.1 LEDP based Core Decomposition

We focus exclusively on *coreness estimation* under local privacy. Let $G = (V, E)$ be an undirected, simple graph with $n = |V|$ vertices and $m = |E|$ edges. For $i \in V$, denote its neighborhood by $N(i)$ and its degree by $\deg(i) = |N(i)|$ (d_u for short). Let $d_{\max} = \max_{i \in V} \deg(i)$.

K-core and core number. For $k \in \mathbb{N}$, the *k-core* of G is the maximal induced subgraph in which every vertex has degree at least k. The *core number* $k(v)$ of a vertex $v \in V$ is the largest k such that v lies in the k-core but not in the $(k+1)$-core.

Table 1. Frequently used notations.

Symbol	Definition
$G = (V, E)$	undirected graph with vertices V and edges E
$n = \|V\|$, $m = \|E\|$	number of vertices and edges
$N(u)$	neighbors of vertex u in G
$d_u = \deg(u)$, $d_{\max}$	degree of u and maximum degree in G
$c_u^{(t)}$	round-t coreness state of u
A_t	active vertex set at round t
$p_t(\cdot)$, H_t	empirical distribution of $\{c_u^{(t)} : u \in A_t\}$ and its entropy
F_t	public noise floor at round t
T	number of iterations
$\varepsilon_{\text{init}}$, $\varepsilon_{\text{prev}}$, $\varepsilon_{\text{used}}$, ε_{rem}, ε_t	initial budget, previous-round budget, used budget, remaining budget, and round-t budget
ε	total privacy budget
$\text{Geom}_{\pm}(\varepsilon)$	symmetric geometric mechanism with parameter ε
$Z(\eta)$	a scalar symmetric geometric noise draw with parameter η, i.e., $Z(\eta) \sim \text{Geom}_{\pm}(\eta)$
$\hat{c}_u$	estimated coreness of u
c_u^*	true coreness of u
$\mathcal{T}(\cdot)$	neighborhood H-index operator

Exact k-core decomposition is incompatible with standard differential privacy guarantees (including local edge-DP). Consequently, we return *approximate* core numbers.

Definition 3.1 ((ϕ, ζ)-approximate core number). Let $\hat{c}_v$ be an estimate of the true core number c_v^*, and let $\phi \geq 1$ and $\zeta \geq 0$. We say $\hat{c}_v$ is a (ϕ, ζ)-approximate core number if

$$c_v^* - \zeta \;\leq\; \hat{c}_v \;\leq\; \phi\, c_v^* + \zeta.$$

ε-edge differential privacy [17]. Let $\mathcal{A}$ be a randomized algorithm that takes a graph $G = (V, E)$ as input and outputs a value in a measurable space $\mathcal{Y}$. Two graphs $G = (V, E)$ and $G' = (V, E')$ on the same vertex set are *edge-neighbors*, written $G \sim_e G'$, if $|E \triangle E'| \leq 1$. We say $\mathcal{A}$ is ε-edge differentially private if for every measurable $S \subseteq \mathcal{Y}$ and all edge-neighboring graphs $G \sim_e G'$,

$$e^{-\varepsilon} \;\leq\; \frac{\Pr[\mathcal{A}(G) \in S]}{\Pr[\mathcal{A}(G') \in S]} \;\leq\; e^{\varepsilon}.$$

Local randomizer for node u. Fix $u \in V$, a randomized map $R_u : 2^V \to \mathcal{Y}$ that takes as input u's neighbor set $a \subseteq V$ (equivalently $a = N(u)$) is *ε-edge DP (locally)* if for every measurable $S \subseteq \mathcal{Y}$ and all a, a' with $|a \triangle a'| \leq 1$,

$$e^{-\varepsilon} \;\leq\; \frac{\Pr[R_u(a) \in S]}{\Pr[R_u(a') \in S]} \;\leq\; e^{\varepsilon}.$$

Probabilities are over the internal randomness of R_u.

Definition 3.2 (Local Edge Differential Privacy (LEDP)). Consider a (possibly interactive) protocol that, on input $G = (V, E)$, has each node $u \in V$ emit a sequence of messages $Y_{u,1}, \ldots, Y_{u,T}$ via local randomizers $R_{u,1}, \ldots, R_{u,T}$. The protocol is ε-LEDP if there exist nonnegative budgets $\varepsilon_{u,1}, \ldots, \varepsilon_{u,T}$ with $\sum_{t=1}^{T} \varepsilon_{u,t} \leq \varepsilon$ for every u, such that for each round t (and conditioning on the past transcript) the map $R_{u,t}$ is $\varepsilon_{u,t}$-edge DP in the sense of the previous definition. By composition and post-processing (see Section 3.3), the joint transcript is then ε-LEDP.

Local DP and relationship DP Our definition follows the standard local edge DP model [30, 31], in which each node protects its own adjacency record independently. For an undirected edge (u, v), the two local records held by u and v are treated as two separate local secrets, and each endpoint receives an ϵ DP guarantee for its own record. If the edge is instead viewed as one shared secret and privacy is measured with respect to the joint outputs of both endpoints, the two local guarantees compose to 2ϵ, which corresponds to relationship DP [30]. This distinction does not affect the LEDP guarantees analyzed in this paper.

LEDP based k-core decomposition [41]. Given G and a privacy budget, output (ϕ, ζ)-approximate core numbers $\{\hat{c}_v\}_{v \in V}$ that satisfy the required local edge-DP constraints in Definition 3.2.

3.2 H-index and Its Relation to Coreness

H-index. For a vertex $u \in V$ with neighbor set $N(u)$, the H-index quantifies how many of u's neighbors are themselves "strong". Given a multiset $S_u = \{ x_v : v \in N(u) \}$ (where x_v is typically $\deg(v)$ or an iterate), the H-index of u is the largest integer

$$H\text{-index}(S_u) \;=\; \max\Big\{ h \in \mathbb{Z}_{\geq 0} :\; |\{ x \in S_u : x \geq h \}| \;\geq\; h \Big\}.$$

It always holds that $0 \leq H\text{-index}(S_u) \leq \deg(u)$, and in practice H-index is more robust than raw degree to occasional very large neighbor values. We use the neighborhood H-index operator $\mathcal{T} : \mathbb{R}^V \to \mathbb{R}^V$,

$$\mathcal{T}_u(x) \;=\; H\text{-index}\big(\{ x_v :\; v \in N(u) \}\big), \qquad u \in V,$$

which acts coordinatewise and maps integer vectors to integer vectors. This view lets us reason about repeated local transformations and their steady-state.

Theorem 3.3 (H-index converges to coreness[4, 37, 45]). *Define the neighborhood H-index operator*

$$\mathcal{T}_u(x) \;=\; H\text{-}index\big(\{ x_v :\; v \in N(u) \}\big), \qquad u \in V,$$

acting coordinatewise on $x \in \mathbb{R}^V$. Initialize the integer vector $x^{(0)}$ by degrees, $x_u^{(0)} = \deg(u)$ for all u, and iterate synchronously

$$x^{(t+1)} \;=\; \mathcal{T}\big(x^{(t)}\big), \qquad t = 0, 1, 2, \ldots$$

Then the sequence $\{x^{(t)}\}_{t \geq 0}$ is integer valued, coordinatewise nonincreasing, and bounded in $[0, \deg(u)]$ for every u. Consequently, it stabilizes in finitely many steps to a fixed point $c^* \in \mathbb{R}^V$ with $c^* = \mathcal{T}(c^*)$. Moreover, for every $u \in V$, the limit value c_u^* equals the coreness (i.e., k-core number) of u.

3.3 Privacy Tools

We summarize the privacy primitives in the *local edge-DP* setting adopted by our algorithms. Unless noted, neighboring inputs refer to adjacency lists that differ by exactly one incident edge. We use "with high probability" (whp) to mean probability at least $1 - n^{-c}$ for any fixed $c \geq 1$.

Definition 3.4 (Global Sensitivity, cf. [17]). Let $f : \mathcal{D} \to \mathbb{R}^d$ and let $\mathcal{D}$ denote the space of adjacency lists. The ℓ_1 global sensitivity of f is

$$\mathrm{GS}(f) = \max_{a,\, a' \text{ edge-neighbors}} \|f(a) - f(a')\|_1 .$$

We add noise using the symmetric geometric distribution—the discrete analogue of the Laplace distribution. Because the H-index updates are integer-valued, this choice integrates naturally with the iteration: it preserves integrality at every step and avoids the rounding and extra arithmetic that continuous noise (e.g., Laplace or Gaussian) would introduce.

Definition 3.5 (Symmetric Geometric Distribution [3, 47]). For $0 < \beta < 1$, the random variable $X \sim \mathrm{Geom}(\beta)$ takes integer values $i \in \mathbb{Z}$ with probability

$$\Pr[X = i] = \frac{1-\beta}{1+\beta}\,\beta^{|i|}.$$

Lemma 3.6 (high-probability bound of Geometric Distribution[33]). *Let $X \sim \mathrm{Geom}(\beta)$ with $0 < \beta < 1$ and $n \geq 2$. Then, for any $c \geq 1$,*

$$\Pr\left[\, |X| \leq \tfrac{c \ln n}{\ln(1/\beta)} \,\right] \geq 1 - n^{-c}.$$

Geometric mechanisms. We show that the geometric mechanism achieves ε-DP by calibrating noise to the ℓ_1-sensitivity.

Let $\mathrm{GS}_1(f) = \max_{a \sim a'} \|f(a) - f(a')\|_1$ be the global ℓ_1-sensitivity of $f : \mathcal{D} \to \mathbb{Z}^d$.

Definition 3.7 (Geometric Mechanism, [7, 23]). Given privacy budget $\varepsilon > 0$ and $f : \mathcal{D} \to \mathbb{Z}^d$ with $\mathrm{GS}_1(f) < \infty$, the mechanism releases

$$\mathcal{M}(a; f, \varepsilon) = f(a) + (Y_1, \ldots, Y_d),$$

where $Y_1, \ldots, Y_d$ are i.i.d. two-sided geometric (discrete Laplace) with parameter

$$\alpha = \exp\Big(-\frac{\varepsilon}{\mathrm{GS}_1(f)}\Big), \qquad \Pr[Y_i = k] = \frac{1-\alpha}{1+\alpha}\,\alpha^{|k|} \quad (k \in \mathbb{Z}).$$

This calibration ensures ε-DP for f under the adjacency $a \sim a'$. (The Laplace mechanism is the continuous analogue for real-valued f.)

Lemma 3.8 (Geometric Mechanism is ε-DP,[3, 17, 18, 41]). *The geometric mechanism satisfies ε-local differential privacy.*

Composition and post-processing. We show that under composition the privacy losses add up, and under post-processing the guarantee is preserved.

Theorem 3.9 (Composition, [16, 17, 19, 41]). *Let $\mathcal{A}_1, \ldots, \mathcal{A}_k$ be mechanisms with privacy parameters $\varepsilon_1, \ldots, \varepsilon_k$. The adaptive composition $(\mathcal{A}_1, \ldots, \mathcal{A}_k)$ is $(\varepsilon_1 + \cdots + \varepsilon_k)$-differentially private.*

Theorem 3.10 (Post-Processing, [6, 17]). *If $\mathcal{M}$ is ε-LDP and h is any (possibly randomized) map, then $h \circ \mathcal{M}$ is also ε-LDP.*

4 Limitations of Existing Works

Coreness estimation under LEDP is substantially more challenging than traditional local graph statistics such as degree or triangle counts. Unlike these tasks, coreness depends on recursively supported subgraph structures rather than local edge counts alone. Existing approaches fall into two main categories, each with fundamental limitations.

RR based methods. Following the CoreRR baseline in KCoreD, we consider an RR reconstruction baseline that applies randomized response to potential edges, reconstructs a noisy graph, and runs standard core decomposition. The graph has $\Theta(n^2)$ non edges but only $\Theta(m)$ true edges, where $m \ll n^2$. Thus, even a small probability of flipping a non edge can create many false positive edges. These edges change local degrees, create artificial neighborhood support, and alter survival under recursive core constraints, destroying the sparsity pattern that determines coreness and substantially distorting core numbers. Explicit reconstruction must also represent randomized outcomes for all vertex pairs rather than only the original $O(m)$ edges, requiring $O(n^2)$ space and high memory and runtime costs on large sparse graphs. Figure 2 illustrates this effect: randomized response densifies the graph, blurs the separation between the high coreness central subgraph and

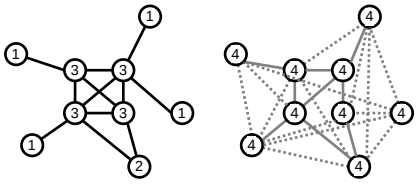


Fig. 2. An example of change after randomized response.

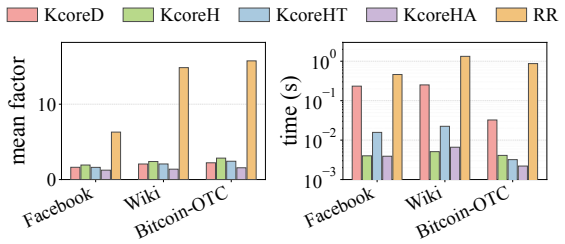


Fig. 3. Accuracy and runtime comparison with randomized response at $\varepsilon = 1$.

its weakly supported periphery, and can assign vertices outside the true core much larger core numbers.

Figure 3 shows that the randomized response baseline is substantially less accurate than all other methods on Facebook, Wiki, and Bitcoin OTC. Its sharply increased mean approximation factors on Wiki and Bitcoin OTC show that false positive edges inflate the estimated core numbers. It is also markedly slower even on these small graphs because it processes a dense perturbed graph over potential vertex pairs. These results confirm that randomized response reconstruction both destroys the sparse structure determining coreness and incurs considerable computational overhead.

Peeling based methods. Peeling based methods [41] approximate classical k-core decomposition by first releasing noisy degree estimates and partitioning vertices into coarse level buckets using noisy threshold comparisons. The algorithm then iteratively refines these level assignments by collecting locally perturbed residual support updates from remaining neighbors and moving vertices across buckets as their estimated residual degrees change, where the highest level at which a vertex remains stable determines its estimated coreness. If a vertex is removed earlier than it should be, its neighbors immediately lose structural support in subsequent rounds and may also be removed incorrectly. Since removed vertices cannot reenter later cores, these early errors accumulate across iterations and affect the final coreness estimates. Moreover, bucketed degree levels only provide discretized structural information and therefore cannot accurately capture small support differences among vertices with similar residual degrees. In addition, the algorithm allocates the privacy budget across multiple peeling rounds and repeatedly communicates intermediate updates as the active vertex set shrinks, so each round operates with reduced signal strength while the total communication cost increases with the number of peeling iterations, making the method less effective on graphs with large coreness depth.

These limitations motivate the need for alternative approaches that avoid brittle peeling decisions and exploit more robust structural relationships between local aggregation and global coreness.

5 H-index-Based Algorithm: KcoreH

In this section, we propose KcoreH, an H-index based iterative algorithm that achieves robust coreness estimation under LEDP without graph reconstruction. The key innovation is to exploit the H-index operator's convergence property: iteratively applying neighborhood H-index aggregation starting from degrees provably converges to the true coreness [37]. This enables us to recover coreness through purely local aggregations with low sensitivity (at most 1), making it naturally suited for LEDP.

Algorithm 1: KcoreH

Input: $G = (V, E)$; privacy budget ε; iterations T.
Output: Core estimator $(\hat{c}_u)_{u \in V}$.

foreach $u \in V$ **do**
  $c_u^{(0)} \leftarrow \max\{0, \deg(u) + \mathrm{Geom}_{\pm}(\varepsilon/T)\}$; ▷ publish $c_u^{(0)}$
for $t = 1$ **to** $T - 1$ **do**
  foreach $u \in V$ **do**
    $h_u \leftarrow H\text{-index}(\{ c_v^{(t-1)} : v \in N(u) \})$;
    $c_u^{(t)} \leftarrow \max\{0, h_u + \mathrm{Geom}_{\pm}(\varepsilon/T)\}$; ▷ publish $c_u^{(t)}$
return $\hat{c}_u \leftarrow \max\{0, c_u^{(T-1)}\}$ *for all* $u \in V$;

5.1 Why H-index for LEDP Coreness

The fundamental challenge in LEDP coreness estimation is that coreness is determined by recursive support relations in the graph, so any accurate estimator must in some form exploit structural information. Under local privacy, however, this information cannot be directly accessed or reliably reconstructed at the edge level. This makes explicit structure-first approaches to coreness estimation fundamentally misaligned with LEDP: the very graph structure one would like to recover is precisely the object that local randomization most strongly obscures. What is needed instead is a mechanism that can use the structural information required for coreness without explicitly reconstructing the graph itself.

The H-index provides exactly such a mechanism. Recall from Section 3 that iteratively applying the neighborhood H-index operator starting from degrees converges to the true coreness (Theorem 3.3). This convergence property is the key reason that H-index is well suited to LEDP coreness estimation. It implies that coreness can be estimated without explicitly recovering the graph structure, because the necessary structural information is exploited implicitly through the iterative local updates themselves. In other words, the recursive support information is not represented as a reconstructed graph or a noisy peeling trajectory, but is progressively encoded in the scalar states propagated across rounds.

This implicit use of structure gives H-index iteration several advantages over peeling-based estimators under LEDP. First, it avoids explicit threshold-based structural decisions on a perturbed graph, which are especially fragile near decision boundaries. Second, because the iteration follows the same recursive support pattern that defines coreness, vertices in strongly supported regions continue to receive strong neighborhood signals across rounds, while weakly supported vertices quickly decay toward smaller values. Third, it avoids the bucket maintenance and repeated threshold testing required by noisy peeling schemes, leading to a more stable refinement process and a simpler communication pattern.

5.2 The KcoreH Algorithm

Algorithm 1 presents the detailed procedure of KcoreH. The algorithm operates in T synchronous rounds, with each vertex maintaining a scalar state $c_u^{(t)}$ that represents its noisy coreness estimate at round t.

Phase 1: Initialization (Lines 1-2). Each vertex u initializes its state by perturbing its degree with geometric noise calibrated to the per round budget ε/T, and publishes this noisy degree $c_u^{(0)}$ to the curator. This initialization provides a starting point that is already correlated with the true coreness, since degree upper-bounds coreness.

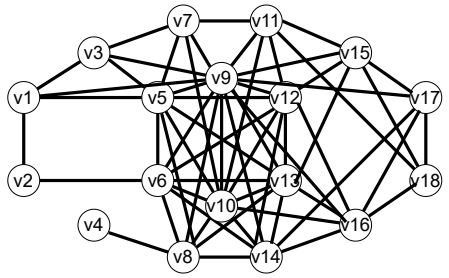


Fig. 4. A running example.

Table 2. The iteration of KcoreH. T is set to 5. RMSE is root mean squared error.

	v1	v2	v3	v4	v5	v6	v7	v8	v9	v10	v11	v12	v13	v14	v15	v16	v17	v18	RMSE	Status
True degree	4	2	4	1	9	8	5	7	14	10	6	9	9	8	6	7	5	4		
True coreness	3	2	3	1	6	6	4	6	6	6	4	6	6	6	4	5	4	4		
KcoreH: $H^{(0)}$	4	3	3	3	7	9	5	5	15	9	6	9	8	7	5	8	5	5	2.769	noisy init
KcoreH: $H^{(1)}$	3	2	3	1	6	6	4	5	8	6	5	6	6	5	4	6	4	5	0.707	refinement
KcoreH: $H^{(2)}$	3	2	3	0	6	5	4	5	7	5	4	6	5	6	4	5	4	4	0.577	essentially converged
KcoreH: $H^{(3)}$	3	2	2	0	5	5	3	4	5	5	4	5	5	5	4	5	4	4	0.882	noise
KcoreH: $H^{(4)}$	3	2	2	0	5	5	3	4	5	5	3	5	5	5	4	4	4	4	0.943	noise

Phase 2: Iterative refinement (Lines 3-6). In each subsequent round $t = 1, \ldots, T-1$, vertex u performs three steps: (i) it retrieves the noisy states $\{c_v^{(t-1)} : v \in N(u)\}$ of its neighbors from the previous round; (ii) it computes the H-index h_u of this multiset, which aggregates the neighborhood information into a single robust statistic; (iii) it perturbs h_u with fresh geometric noise and publishes the updated state $c_u^{(t)}$. This iterative process mimics the noiseless H-index iteration that converges to coreness, but under LEDP each round injects fresh noise to protect the neighborhood structure.
Phase 3: Output (Line 7). After T rounds, the algorithm returns the final noisy state $\hat{c}_u = c_u^{(T-1)}$ as the coreness estimate for each vertex u. The privacy budget is uniformly split across rounds, so each release consumes exactly ε/T budget.
Key design choices. The algorithm uses the symmetric geometric mechanism because H-index values are integers, making discrete noise natural and avoiding rounding artifacts. The uniform budget split is simple and ensures that no single round dominates the privacy loss. The H-index aggregation has sensitivity 1 (adding or removing one neighbor changes the H-index by at most 1), which means the noise magnitude is directly controlled by the per round budget ε/T.
Choosing the number of rounds. Choosing T balances H-index refinement and privacy noise. More rounds reduce the finite round residual, but they also reduce the per round privacy budget to ε/T, increasing the noise introduced in every release and accumulating noise over more rounds. The analysis below formalizes this tradeoff, while Section 8 evaluates it empirically and provides practical guidance for choosing T on new graphs.
A running example. Table 2 illustrates the noisy iteration of KcoreH on the graph with 18 vertices shown in Figure 4. The row $H^{(0)}$ is obtained by perturbing the true degrees with geometric noise and has RMSE 2.769. After one round of noisy H index refinement, the RMSE drops to 0.707, and many vertices match their true coreness values. At $H^{(2)}$, the RMSE further decreases to 0.577,

the best among all rounds shown, and the number of red entries representing incorrect estimates is also the smallest. Thus, the useful refinement is essentially complete within two iterations.

Continuing beyond this point does not improve the estimates, as newly injected noise begins to dominate the updates. At $H^{(3)}$ and $H^{(4)}$, more vertices deviate from their true coreness, and the RMSE increases to 0.882 and 0.943, respectively. Once the estimates have nearly converged, additional noisy iterations mainly introduce fresh perturbations. This motivates adaptive stopping, since a fixed iteration budget may continue after the effective refinement stage has ended.

5.3 Theoretical Analysis

We now analyze the privacy and accuracy guarantees of KcoreH. Our analysis exploits two key properties of the H-index operator: (i) it is monotone and nonexpansive (1-Lipschitz), which bounds error propagation across rounds; and (ii) it has low sensitivity (at most 1), which enables tight privacy-utility trade-offs under geometric noise.

We analyze KcoreH on an undirected simple graph $G = (V, E)$, and let $c^* = (c^*_u)_{u\in V} \in \mathbb{N}^V$ be the coreness vector characterized by $c^* = \mathcal{T}(c^*)$, where

$$(\mathcal{T}(x))_u = H\text{-index}\big(\{\, x_v : v \in N(u) \,\}\big), \qquad u \in V.$$

Let $\varepsilon_r := \varepsilon/T$ and $\alpha := e^{-\varepsilon_r}$. The algorithm maintains $c^{(t)} = (c^{(t)}_u)_{u\in V}$ for $t = 0, 1, \ldots, T-1$, and the released estimator is $\hat{c} = (\hat{c}_u)_{u\in V} = c^{(T-1)}$. Write $e^{(t)} := c^{(t)} - c^*$ and $\|x\|_\infty := \max_{u\in V} |x_u|$.

$$c^{(0)} = [\, d + \xi^{(0)} \,]_+, \tag{1}$$

where $\xi^{(0)} = (Z_{u,0})_{u\in V}$ and each coordinate follows $\mathrm{Geom}_\pm(\varepsilon_r)$.

$$c^{(t)} = \big[\, \mathcal{T}\big(c^{(t-1)}\big) + \xi^{(t)} \,\big]_+ \qquad (t = 1, \ldots, T-1), \tag{2}$$

where $\xi^{(t)} = (Z_{u,t})_{u\in V}$.

Privacy guarantee. The privacy analysis is straightforward due to the uniform budget split and unit sensitivity of H-index.

Theorem 5.1 (Privacy of KcoreH). *KcoreH satisfies edge local differential privacy.*

Proof. In KcoreH, each local release is obtained by adding symmetric geometric noise to a quantity with local edge sensitivity at most one.

First, the initialization releases $c^{(0)}_u = \max\{0, \deg(u) + \mathrm{Geom}_\pm(\varepsilon/T)\}$, which is (ε/T)-LEDP by Lemma 3.8, since the degree changes by at most one under the local edge adjacency model.

Then, at each round $t = 1, \ldots, T-1$, vertex u releases

$$c^{(t)}_u = \max\big\{0, \mathcal{T}(c^{(t-1)})_u + \mathrm{Geom}_\pm(\varepsilon/T)\big\}.$$

The neighborhood H-index operator $\mathcal{T}(\cdot)_u$ also has local edge sensitivity at most one under the local edge adjacency model, so each such release is again (ε/T)-LEDP by Lemma 3.8. The final output is computed from the released noisy values, and therefore incurs no additional privacy loss by the post-processing property in Theorem 3.10.

Since KcoreH uses exactly T local releases per vertex, each with privacy cost ε/T, the total privacy loss is $T \cdot \frac{\varepsilon}{T} = \varepsilon$ by Theorem 3.9. Hence KcoreH is ε-LEDP. □

Accuracy guarantees: finite-round H-index refinement. The key to our accuracy analysis is that the H-index operator is nonexpansive. Noise introduced in one round is not amplified by subsequent updates, so the perturbations accumulate additively rather than multiplicatively.

To separate the effect of privacy noise from the error caused by running only finitely many H-index updates, let $x^{(0)} = d$ and define the noiseless trajectory $x^{(t)} = \mathcal{T}(x^{(t-1)})$ for $t \geq 1$. For a

horizon T, define $B_T(G) := \|x^{(T-1)} - c^*\|_\infty$. The quantity $B_T(G)$ is the deterministic error remaining after $T-1$ noiseless H-index updates. It replaces the previous coarse dependence on $d_{\max}$ by the actual finite-round residual of the H-index recursion.

The following stability properties are used in both accuracy results.

Lemma 5.2 (Monotonicity and nonexpansiveness). *For $x, y \in \mathbb{Z}^V$, if $x \le y$, then $\mathcal{T}(x) \le \mathcal{T}(y)$. Moreover, $\|\mathcal{T}(x) - \mathcal{T}(y)\|_\infty \le \|x - y\|_\infty$.*

Lemma 5.3 (Nonexpansiveness of truncation). *For $P(x) := [x]_+$, $\|P(x) - P(y)\|_\infty \le \|x - y\|_\infty$.*

Theorem 5.4 (High-probability finite-round accuracy). *For every $T \ge 1$, the output of KcoreH satisfies*

$$\|\hat{c} - c^*\|_\infty \le B_T(G) + \sum_{t=0}^{T-1} \|\xi^{(t)}\|_\infty.$$

For any $\delta \in (0,1)$, let

$$b_\delta := \left\lceil \frac{T}{\varepsilon} \ln\left(\frac{2nT}{\delta}\right) \right\rceil.$$

Then, with probability at least $1-\delta$,

$$\max_{u \in V} |\hat{c}_u - c^*_u| \le B_T(G) + T b_\delta. \tag{3}$$

Equivalently, $\hat{c}$ is a factor-one approximate core-number estimate with additive error $\zeta_{T,\delta} = B_T(G) + T b_\delta$.

What this bound tells us. Theorem 5.4 separates two sources of error. The term $B_T(G)$ measures how far the noiseless H-index trajectory remains from the exact core-number vector after $T-1$ updates. The term Tb_δ controls the privacy noise simultaneously over all vertices and all releases. Nonexpansiveness prevents this noise from being amplified by later H-index updates. In particular, if the noiseless trajectory converges within $T-1$ updates, then $B_T(G) = 0$ and the only remaining error is due to privacy noise.

Theorem 5.5 (Expected finite-round accuracy). *For every $T \ge 1$, there exists an absolute constant $C > 0$ such that*

$$\mathbb{E}\,\|\hat{c} - c^*\|_\infty \le B_T(G) + C\,\frac{T^2 \ln(2n)}{\varepsilon}.$$

What this bound tells us. This theorem controls the expected maximum error over all vertices. One factor of T comes from dividing the privacy budget across T releases, which gives per-round noise of scale T/ε. The second factor comes from accumulating the perturbations from the T releases. The result therefore makes explicit the tradeoff in the number of rounds: increasing T reduces the finite-round residual $B_T(G)$ but increases the accumulated noise.

When does the approximation guarantee become informative? Theorem 5.4 gives a factor-one approximation with $\zeta_{T,\delta} = B_T(G) + Tb_\delta$. For every vertex with $c^*_u > 0$,

$$\frac{|\hat{c}_u - c^*_u|}{c^*_u} \le \frac{B_T(G) + Tb_\delta}{c^*_u}.$$

Thus, the relative error of vertex u is at most η whenever $c^*_u \ge (B_T(G) + Tb_\delta)/\eta$.

The finite-round residual can be related to several structural properties of the graph. Define

$$\tau_H(G) := \min\{t \ge 0 : x^{(t)} = c^*\}, \qquad g(G) := \max_{u \in V}(d_u - c^*_u),$$

$$\mu(G) := \max_{\emptyset \ne U \subseteq V} \frac{2|E(U)|}{|U|}, \qquad \kappa(G) := \max_{u \in V} c^*_u.$$

These quantities bound the finite-round residual as follows:

$$B_T(G) \le \min\{g(G), \lfloor \mu(G) n^{1/T} \rfloor\} \le \min\{g(G), \lfloor 2\kappa(G) n^{1/T} \rfloor\}.$$

In particular, $\tau_H(G) \le T-1$ implies $B_T(G) = 0$. A limited degree spread within each connected component also gives a correspondingly small value of $g(G)$. Hence, the residual is small when the noiseless H-index trajectory stabilizes rapidly, when degrees are close to core numbers, or when the graph is sufficiently sparse relative to the chosen horizon T.

For fixed T and $\delta = n^{-c}$, $Tb_\delta = O(\ln n/\varepsilon)$. Therefore, on graph classes satisfying $B_T(G) = O(\ln n/\varepsilon)$, Theorem 5.4 gives $\big(1, O(\ln n/\varepsilon)\big)$-approximate core numbers. For the usual regime of constant ε, this condition includes bounded-degree graphs, such as road and grid-like infrastructure networks, sparse graphs with limited degree spread, and graphs whose noiseless H-index trajectories stabilize within $T-1$ updates. Sparse graphs with high-degree hubs may also satisfy the condition when the hub degrees are not supported by sufficiently many high-coreness neighbors and are therefore reduced rapidly by H-index refinement. The density-based bound further establishes the condition whenever $\mu(G) n^{1/T} = O(\ln n/\varepsilon)$.

By comparison, KcoreD gives $\big(2+\eta_{\mathrm{D}}, O(\ln d_{\max} \ln^2 n/\varepsilon)\big)$-approximate core numbers. Hence, on graph classes for which finite H-index refinement leaves a small residual, KcoreH removes the multiplicative distortion in the KcoreD guarantee and has a smaller additive dependence on graph size.

We audit the finite-round residual on four real-world graphs. From $T=1$ to $T=5$, $B_T(G)$ decreases from 975 to 10 on Facebook, from 1012 to 21 on Wiki, from 5412 to 67 on Flickr, and from 19375 to 214 on Google+. Thus, although the initial degrees can be much larger than the core numbers, a small number of H-index updates removes most of this initial discrepancy. At $T=5$, the remaining residual is 8.7%, 39.6%, 11.7%, and 28.5% of the maximum core number on Facebook, Wiki, Flickr, and Google+, respectively. In addition, the 95th-percentile perturbation reaching the final output is only 20.2%–24.5% of the worst-case accumulated-noise bound. Hence, the finite-round residual and the propagation of privacy noise are substantially reduced relative to their initial or worst-case values, although the residual remains graph-dependent and non-negligible on some power-law graphs. On Flickr and Google+, the formal certificates are 69.1% and 72.2% of the maximum core number, while the realized 95th-percentile maximum errors are only 16.2% and 32.6%, respectively, consistent with the low estimation errors reported in Section 8.

Although obtaining a tight uniform bound for coreness estimation is inherently difficult because it must hold with probability $1-\delta$ simultaneously for all n vertices and all T releases, thereby incurring an $O(\ln n)$ factor relative to the typical error of one vertex, our bound with multiplicative factor one is nevertheless substantially tighter than the KcoreD guarantee, which has multiplicative factor $(2+\eta_{\mathrm{D}})$ and a larger additive term.

Complexity analysis. Computationally, KcoreH performs an H-index bucket counting pass per round, resulting in $O(m)$ work per iteration and $O(T\,m)$ end to end time. Its memory footprint consists of a length-n state vector plus per node scratch proportional to neighborhood size, giving $O(n+m)$ total. For communication cost, each vertex uploads one scalar per round and must receive updated neighbor states, yielding uplink $O(T\,n)$ and downlink $O(T\,m)$.

6 Transition Based H-index Algorithm: KcoreHT

The KcoreH algorithm in Section 5 produces robust coreness estimates by iteratively refining H-index aggregations under LEDP. However, KcoreH suffers from a fundamental inefficiency: it updates every vertex in every round under a fixed iteration budget, even after many vertices have already converged. As shown in the running example (Table 2), different vertices stabilize at different speeds: low and medium degree vertices often converge within 2-3 rounds, while only a

smaller subset of structurally difficult vertices continues to change. Yet KcoreH forces all vertices to absorb fresh noise in every round, causing already converged vertices to drift and accumulate unnecessary error. In this section, we propose KcoreHT, a transition based variant of KcoreH that addresses this inefficiency. The key innovation is a local locking mechanism: vertices with sufficiently small inter round changes are frozen early, and only the remaining unstable vertices continue to participate in later rounds. This reduces unnecessary noisy updates while preserving the simplicity of iterative H-index refinement. We first present the algorithm, then analyze its privacy and complexity guarantees.

6.1 Transition based Locking Mechanism

The core idea of KcoreHT is to lock vertices whose changes are small relative to the noise scale. At round t, it sets $\tau = 2\,\mathbb{E}|\mathrm{Geom}_{\pm}(\varepsilon_r)|$, twice the expected magnitude of the geometric noise. If $|c_u^{(t)} - c_u^{(t-1)}| \le \tau$, vertex u is considered stable and retains $c_u^{(t-1)}$ in all subsequent rounds; otherwise, it remains active for further refinement. This conservative rule avoids locking vertices that still exhibit changes beyond the noise scale, while capturing most converged vertices within a few rounds, thereby reducing noisy updates and improving accuracy.

Algorithm 2 presents the detailed procedure of KcoreHT. Like KcoreH, the algorithm runs for T synchronous rounds and maintains a scalar state $c_u^{(t)}$ for each vertex u. In addition, it maintains an active set A_t that records the vertices whose estimates are still being updated.

Phase 1: Initialization (Lines 1–5). As in KcoreH, each vertex perturbs and publishes its degree, and all vertices enter the initial active set $A_0 = V$.

Phase 2: Transition-aware refinement (Lines 6–17). At round t, each vertex in A_{t-1} computes and perturbs its neighborhood H-index using the previous neighbor states. From the released estimates, the curator determines whether the change is at most τ. If so, the vertex retains its previous estimate and becomes inactive; otherwise, it updates its state and remains active. Inactive vertices retain their estimates without further releases.

Phase 3: Output (Line 18). After T rounds, the curator returns $\hat{c}_u = \max\{0, c_u^{(T-1)}\}$ for each vertex u.

6.2 Theoretical Analysis

Privacy guarantee. We now show that KcoreHT satisfies edge local differential privacy.

Theorem 6.1 (Privacy of KcoreHT). *KcoreHT satisfies ε edge local differential privacy.*

Proof. Each vertex makes at most T releases: one noisy degree and at most $T-1$ noisy H-index values. Each has local edge sensitivity 1 and is (ε/T) LEDP under the geometric mechanism. Composition gives total privacy loss at most ε, while active set updates, locking, and the final output preserve privacy by post processing. □

Complexity analysis. In the worst case, KcoreHT has the same asymptotic complexity as KcoreH. Each round performs one neighborhood H-index update per active vertex, yielding $O(m)$ work per iteration and $O(Tm)$ total time. Maintaining the active set adds only $O(n)$ work per round. The total space complexity is $O(n+m)$. For communication, the worst case uplink is $O(Tn)$ and the downlink is $O(Tm)$.

7 Budget and Stopping Adaptive H-index Algorithm: KcoreHA

Although KcoreHT reduces unnecessary updates by locking stable vertices, both KcoreH and KcoreHT require a fixed iteration budget T. Choosing T requires prior knowledge and careful tuning: too few rounds prevent convergence, while too many waste privacy budget and accumulate

Algorithm 2: KcoreHT

Input: $G = (V, E)$; privacy budget ε; iterations T.
Output: Core estimator $(\hat{c}_u)_{u \in V}$.

$\varepsilon_r \leftarrow \varepsilon / T$;
$\tau \leftarrow 2\,\mathbb{E}|\mathrm{Geom}_{\pm}(\varepsilon_r)|$;
foreach $u \in V$ **do**
  $c_u^{(0)} \leftarrow \max\{0,\ \deg(u) + \mathrm{Geom}_{\pm}(\varepsilon_r)\}$;
$A_0 \leftarrow V$;
for $t = 1$ **to** $T - 1$ **do**
  $A_t \leftarrow \emptyset$;
  foreach $u \in V$ **do**
    if $u \in A_{t-1}$ **then**
      $\widetilde{c}_t(u) \leftarrow \max\{0,\ H\text{-index}(\{\, c_v^{(t-1)} : v \in N(u) \,\}) + \mathrm{Geom}_{\pm}(\varepsilon_r)\}$;
      if $|\widetilde{c}_t(u) - c_u^{(t-1)}| \le \tau$ **then**
        $c_u^{(t)} \leftarrow c_u^{(t-1)}$;
      else
        $c_u^{(t)} \leftarrow \widetilde{c}_t(u)$;
        $A_t \leftarrow A_t \cup \{u\}$;
    else
      $c_u^{(t)} \leftarrow c_u^{(t-1)}$;
return $\hat{c}_u \leftarrow \max\{0,\ c_u^{(T-1)}\}$ *for all* $u \in V$;

noise. Moreover, without a graph level convergence criterion, both methods continue after the graph has stabilized. To address these limitations, we propose KcoreHA, which (i) allocates smaller round budgets when the state entropy is high to allow further refinement, and larger budgets as entropy decreases; (ii) stops automatically when the empirical state distribution indicates graph level convergence; and (iii) locks stable vertices while continuing to refine difficult ones. KcoreHA therefore determines the effective number of rounds online without prior graph knowledge or unnecessary iterations. We next present the algorithm and analyze its privacy and complexity.

7.1 Adaptive Allocation and Stopping Criterion

The core innovation of KcoreHA is a two level adaptive mechanism that jointly optimizes privacy budget allocation and stopping time.

Adaptive budget allocation. At each round t, KcoreHA computes the empirical distribution $p_{t-1}(\cdot)$ of the current active states $\{c_u^{(t-1)} : u \in A_{t-1}\}$ and their entropy $H_{t-1} = -\sum_a p_{t-1}(a) \log_2 p_{t-1}(a)$. The entropy measures the diversity of the current estimates: high entropy indicates that estimates are still dispersed and refinement is ongoing, while low entropy indicates that estimates have concentrated around a few values and convergence is near. We then allocate the round budget as

$$\varepsilon_t = \frac{\varepsilon - \varepsilon_{\text{used}}}{1 + H_{t-1}}$$

where $\varepsilon_{\text{used}}$ is the total budget consumed so far. This allocation has two key properties: (i) when entropy is high (early rounds with diverse estimates), we allocate a smaller per round budget, allowing more rounds of refinement; (ii) when entropy is low (late rounds with concentrated estimates), we allocate a larger per round budget, quickly exhausting the remaining budget and

stopping iteration. This ensures that we spend privacy budget efficiently: more budget on refinement when it is effective, less when estimates have stabilized.

Graph level stopping criterion. After each round, we update the active set A_t by retaining only those vertices whose public change $|c_u^{(t)} - c_u^{(t-1)}|$ exceeds the noise floor $F_t = \mathbb{E}|Z(\varepsilon_{\text{prev}})| + \mathbb{E}|Z(\varepsilon_t)|$, where $\varepsilon_{\text{prev}}$ and ε_t are the previous and current round budgets. If $A_t = \emptyset$, we stop iteration immediately, as no vertex has changed significantly. This provides a graph level convergence signal that complements the vertex level locking: even if individual vertices appear stable, we continue iterating until the entire graph has converged.

Motivation for empirical distribution. We use the empirical distribution of the current estimates as a graph level convergence signal. This is motivated by the fact that coreness is a collective structural quantity: if a vertex has coreness k, then it belongs to a k-core and is supported by many other vertices whose coreness values are also at least k. In particular, a vertex with large coreness cannot appear as an isolated outlier, since a coreness value of k implies the presence of at least k other vertices whose coreness values are also at least k. As a result, once the estimates become close to convergence, their empirical distribution tends to stabilize across rounds instead of fluctuating arbitrarily. This makes distributional change a natural criterion for deciding whether further iterations are still useful.

Algorithm 3 presents the detailed procedure of KcoreHA. Unlike KcoreH and KcoreHT, KcoreHA does not fix the number of refinement rounds in advance. Instead, it adaptively allocates privacy budget across rounds and maintains an active set A_t of vertices whose estimates have not yet stabilized.

Phase 1: Initialization (Lines 1–5). The algorithm first allocates an initial budget $\varepsilon_{\text{init}}$, perturbs each degree with geometric noise, and publishes the resulting initial state $c_u^{(0)}$. All vertices are initially placed in the active set $A_0 = V$, and the algorithm records the used and previous-round budgets for later adaptive updates.

Phase 2: Adaptive refinement (Lines 6–20). In each round, KcoreHA computes the empirical distribution of the current active estimates and its entropy, and uses this entropy to determine the next round budget ε_t. It then updates only the active vertices by applying a noisy neighborhood H-index step, while inactive vertices keep their previous values. After the update, the new active set is defined by comparing each active vertex's change against the public noise floor F_t: vertices with changes below this threshold are removed from further interaction, while the others remain active. The process continues until either the active set becomes empty or the consumed budget reaches the total budget ε.

Phase 3: Final output (Lines 21–28). If there is remaining budget $\varepsilon_{\text{rem}} > 0$, the algorithm uses it for one final noisy H-index refinement to produce $\hat{c}_u$; otherwise, it directly outputs the latest state as the final coreness estimate.

7.2 Theoretical Analysis

Privacy guarantee. We now show that KcoreHA satisfies edge local differential privacy.

Theorem 7.1 (Privacy of KcoreHA). *Let T be the number of rounds actually executed in Algorithm 3. Then KcoreHA satisfies edge local differential privacy. For every realized public transcript,*

$$\varepsilon_{\text{init}} + \sum_{t=1}^{T} \varepsilon_t + \varepsilon_{\text{rem}} = \varepsilon.$$

Proof. The initialization release $c_u^{(0)} \leftarrow [\, \deg(u) + \text{Geom}_{\pm}(\varepsilon_{\text{init}}) \,]_+$ is $\varepsilon_{\text{init}}$-LDP by the same sensitivity-1 argument as in Theorem 5.1.

Algorithm 3: KcoreHA

Input: $G = (V, E)$; privacy budget ε.
Output: Core estimator $(\hat{c}_u)_{u \in V}$.

$\varepsilon_{\text{init}} \leftarrow \varepsilon / \big(1 + \max\{1, \log_2(1+n)\}\big)$;
foreach $u \in V$ **do**
  $c_u^{(0)} \leftarrow \max\{0, \deg(u) + \text{Geom}_{\pm}(\varepsilon_{\text{init}})\}$;
$A_0 \leftarrow V$; $t \leftarrow 1$;
$\varepsilon_{\text{used}} \leftarrow \varepsilon_{\text{init}}$; $\varepsilon_{\text{prev}} \leftarrow \varepsilon_{\text{init}}$;
while $A_{t-1} \neq \emptyset$ ***and*** $\varepsilon_{\text{used}} < \varepsilon$ **do**
  Let $p_{t-1}(\cdot)$ be the empirical distribution of $\{c_u^{(t-1)} : u \in A_{t-1}\}$;
  $H_{t-1} \leftarrow -\sum_a p_{t-1}(a) \log_2 p_{t-1}(a)$;
  $\varepsilon_t \leftarrow (\varepsilon - \varepsilon_{\text{used}})/(1 + H_{t-1})$;
  $F_t \leftarrow \mathbb{E}|Z(\varepsilon_{\text{prev}})| + \mathbb{E}|Z(\varepsilon_t)|$;
  foreach $u \in V$ **do**
    if $u \in A_{t-1}$ **then**
      $c_u^{(t)} \leftarrow \max\{0, H\text{-index}(\{c_v^{(t-1)} : v \in N(u)\}) + \text{Geom}_{\pm}(\varepsilon_t)\}$;
    else
      $c_u^{(t)} \leftarrow c_u^{(t-1)}$;
  $A_t \leftarrow \{u \in A_{t-1} : |c_u^{(t)} - c_u^{(t-1)}| > F_t\}$;
  $\varepsilon_{\text{used}} \leftarrow \varepsilon_{\text{used}} + \varepsilon_t$; $\varepsilon_{\text{prev}} \leftarrow \varepsilon_t$;
  if $A_t = \emptyset$ **then**
    break;
  $t \leftarrow t + 1$;
$\varepsilon_{\text{rem}} \leftarrow \max\{0, \varepsilon - \varepsilon_{\text{used}}\}$;
if $\varepsilon_{\text{rem}} > 0$ **then**
  foreach $u \in V$ **do**
    $\hat{c}_u \leftarrow \max\{0, H\text{-index}(\{c_v^{(t)} : v \in N(u)\}) + \text{Geom}_{\pm}(\varepsilon_{\text{rem}})\}$;
else
  foreach $u \in V$ **do**
    $\hat{c}_u \leftarrow c_u^{(t)}$;
return $(\hat{c}_u)_{u \in V}$;

At each later round $t \geq 1$, every active vertex $u \in A_{t-1}$ releases

$$c_u^{(t)} \leftarrow \Big\lfloor H\text{-index}(\{c_v^{(t-1)} : v \in N(u)\}) + \text{Geom}_{\pm}(\varepsilon_t) \Big\rfloor_+,$$

which is ε_t-LDP by the same neighborhood-H-index sensitivity argument as in Theorem 5.1. Inactive vertices release nothing and incur no privacy loss.

The active sets A_t, the empirical distributions $p_t(\cdot)$, the entropies H_t, the round budgets ε_t, the thresholds F_t, and the freezing decisions are all computed from previously released public values, and thus incur no additional privacy loss by post processing.

If $\varepsilon_{\text{rem}} > 0$, the final refinement step is one additional noisy neighborhood-H-index release with budget ε_{rem}, and is therefore ε_{rem}-LDP by the same argument. Otherwise, the output is pure post processing.

Table 3. Communication cost and time complexity.

Method	Communication Cost	Time Complexity
KcoreD [41]	$O(n \log n \log d_{\max})$	$O(m \log n \log d_{\max})$
KcoreH	$O(T(n+m))$	$O(Tm)$
KcoreHT	$O(T(n+m))$	$O(Tm)$
KcoreHA	$O(T(n+m))$	$O(Tm)$

Hence, for each vertex u,

$$\varepsilon_{\text{init}} + \sum_{t \geq 1: u \in A_{t-1}} \varepsilon_t + \varepsilon_{\text{rem}} \leq \varepsilon_{\text{init}} + \sum_{t=1}^{T} \varepsilon_t + \varepsilon_{\text{rem}} = \varepsilon.$$

By composition, KcoreHA is ε-LEDP. □

Complexity analysis. Table 3 summarizes the communication cost and time complexity of all compared methods. In the worst case, KcoreHA has the same asymptotic complexity as KcoreH. Each round performs one neighborhood H-index update per active vertex, yielding $O(m)$ work per iteration and $O(Tm)$ total time. Maintaining the active set and the empirical state distribution adds only $O(n)$ work per round and does not change the leading order. The total space complexity is $O(n+m)$. For communication, the worst case uplink is $O(Tn)$ and the downlink is $O(Tm)$. In practice, early freezing and adaptive stopping can further reduce the actual communication cost.

8 Experimental evaluation

In this section, we evaluate our coreness estimation algorithms under LEDP through experiments.

8.1 Experimental Settings

Datasets. Table 4 summarizes 11 main SNAP datasets and two auxiliary datasets: Bitcoin-OTC for the CoreRR comparison in Section 4 and T sensitivity study, and DBLP for distributed emulation.
Algorithms. We evaluate the following coreness estimation algorithms under edge LDP.
(1) *KcoreD*: the distributed coreness algorithm under edge LDP proposed in [41], which adds geometric noise to each node's degree to determine a private iteration threshold, then iteratively updates levels using noisy same level neighbor counts;
(2) *KcoreH*: our H-index based coreness estimation algorithm;
(3) *KcoreHT*: KcoreH with early termination optimization;
(4) *KcoreHA*: KcoreH with adaptive convergence detection.

As discussed in Section 4, the RR-based baseline performs poorly even on small graphs because false-positive edges substantially distort the graph structure and make the reconstructed graph much denser. We therefore use it only as a diagnostic baseline, rather than as a full-scale competitor.

All algorithms are implemented in C++. The experiments are run on a MacBook Pro with an Apple M4 Pro processor and 48GB memory. To avoid confounding effects from multithreading, we restrict all runs to a single CPU core.
Parameter settings. By default, the privacy budget ε is set to 1. We also allow it to vary from 0.25 to 1.5 with increments of 0.25. For KcoreD, we follow the parameter configuration used in [41]: 80 worker processes, bias term 8, approximation factor 5.625, and privacy split 0.8 for the degree upload stage. As defined in Section 3.1, standard ε-edge LDP protects each endpoint's record

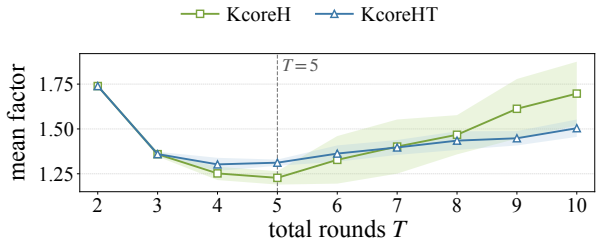


Fig. 5. Sensitivity to the number of rounds T at $\varepsilon = 1$, averaged over Facebook, Wiki, and Bitcoin-OTC. Shaded regions show standard deviations over 20 seeds.

Table 4. Summary of datasets.

Dataset	$\|V\|$	$\|E\|$	d_{avg}	$d_{\max}$
facebook	4 039	88 234	43.69	1K
wiki	7 115	103 689	29.15	1167
flickr	105 938	2 316 948	43.74	5425
gplus	107 614	12 238 285	227.45	20 127
youtube	1 157 828	2 987 624	5.16	29K
roadnetca	1 957 027	2 760 388	2.82	12
wikiedges	2 394 386	4 659 565	3.89	100029
orkut	3 072 441	117 185 083	76.28	33K
livejournal	4 847 573	68 993 774	28.47	22889
twitter	41 652 230	1 468 365 182	70.51	3081112
friendster	65 608 366	1 806 067 135	55.06	5214
bitcoin otc	5 881	21 492	7.31	795
dblp	317 080	1 049 866	6.62	343

independently, whereas ε-relationship DP [30] treats an edge as one shared secret and bounds the sum of both endpoints' privacy losses by ε. KcoreD adopts the latter definition [41] and allocates half of its input budget to each endpoint. To match the same per-endpoint budget ε used by our methods, we therefore run KcoreD with input budget 2ε, which equals 2 under the default setting. For KcoreH and KcoreHT, we set $T = 5$ rounds. Figure 5 shows the average sensitivity to T. In the separate results for Facebook, Wiki, and Bitcoin-OTC, KcoreH achieves its best accuracy at $T = 5$, 6, and 5, respectively, while KcoreHT achieves its best accuracy at $T = 4$ on all three graphs. These results support $T = 5$ as a robust common setting. For a new graph, we recommend $T = 5$ for both methods when $|V| < 10^6$. When $|V| \geq 10^6$, we heuristically extrapolate from these results and suggest $T = 6$ as a practical starting point. Density and degeneracy can further refine this choice, with larger values favoring the upper setting. For KcoreHA, we do not introduce additional hand tuned parameters.

We use KcoreD [41] as the main external baseline because it is the most recent dedicated LEDP k-core estimator. Moreover, [41] already reports direct empirical comparisons against the earlier LEDP k-core method of Dhulipala et al. [10], so we do not repeat that comparison here.

8.2 Efficiency Evaluation

We first evaluate the efficiency of our methods and the compared baseline. Figure 6 plots the end to end runtimes on a logarithmic scale of our three approaches, KcoreH, KcoreHT, and KcoreHA,

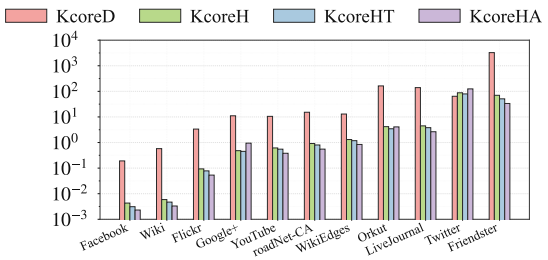


Fig. 6. Average runtime over five runs.

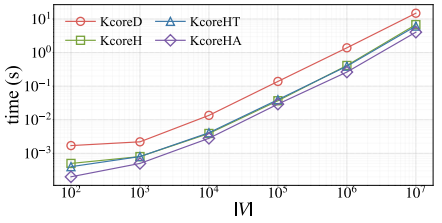


Fig. 7. Scalability test on nested Twitter subgraphs.

against the distributed baseline. Our LEDP methods keep the per round work local and lightweight, whereas the baseline repeatedly sweeps over partitions. The gap is already substantial on medium sized graphs such as *Facebook*, *Wiki*, and *Google+*, where the runtime drops from about 10^{-1} to 10^{1} seconds for the baseline to roughly 10^{-3} to 10^{0} seconds for our methods. It becomes even more pronounced on large graphs such as *Orkut*, *LiveJournal*, and *Friendster*, where the baseline requires 162.6, 139.6, and 3.26×10^{3} seconds, while our methods finish in only 3.5 to 4.2, 2.6 to 4.5, and 33.6 to 70.0 seconds, respectively. Our methods are much faster on most datasets because they use lightweight local H-index updates and stop after a small number of rounds, whereas KcoreD repeatedly scans graph partitions and performs more expensive refinement steps. Twitter is the only dataset on which KcoreD is faster than our methods. This is mainly because Twitter contains approximately 1.47×10^{9} edges, making each neighborhood H-index update substantially more expensive. In addition, KcoreHA runs adaptively until convergence and therefore requires more rounds on Twitter than on the other datasets. The combination of higher per-round cost and more iterations explains its different runtime pattern. Overall, these results show that local LEDP estimation is typically much more efficient than repeated partition sweeps, especially on large graphs.

Comparing our three LEDP variants, the runtimes stay within a relatively narrow band but still exhibit a clear pattern. On small and medium graphs such as *Facebook*, *Wiki*, *Flickr*, *YouTube*, and *roadNet-CA*, all three finish within about 10^{-3} to 10^{0} seconds, and the differences remain modest. The gap widens on the largest and densest graphs. On *Orkut*, KcoreH takes about 4.2 seconds, KcoreHT about 3.5 seconds, and KcoreHA about 4.1 seconds. On *LiveJournal*, the corresponding runtimes are about 4.5, 3.8, and 2.6 seconds. On *Friendster*, they are about 70.0, 50.8, and 33.6 seconds.

Overall, KcoreHA is often the fastest on the heaviest graphs, while KcoreH and KcoreHT remain competitive and consistently far below the baseline.

Scalability. We sample nested induced subgraphs from the SNAP Twitter graph, with $n \approx 4.17 \times 10^{7}$ and $m \approx 1.47 \times 10^{9}$, at sizes $10^{2}, 10^{3}, \ldots, 10^{7}$. As shown in Figure 7, using one fixed random seed, all methods observe identical instances. For each subgraph, we execute KcoreD, KcoreH, KcoreHT, and KcoreHA under the same parameter settings and record end to end runtime. All three algorithms exhibit near linear scaling in log-log space, showing that our LDP estimators remain practical even on billion edge networks.

Communication cost. Figure 8 reports the average number of uploads per vertex over five runs on the eleven datasets. KcoreD requires substantially more communication, especially on large graphs. It uses about 53.4, 30.9, 44.1, and 40.0 uploads per vertex on *Orkut*, *LiveJournal*, *Twitter*, and *Friendster*, respectively, whereas our methods require only about 2.5 to 5.0. KcoreH uses exactly five uploads per vertex because it executes five fixed rounds. KcoreHT has the lowest

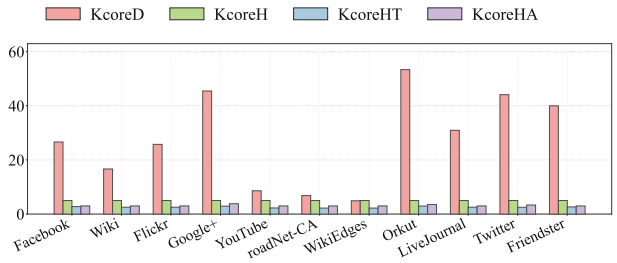


Fig. 8. Number of uploads per vertex, averaged over five runs.

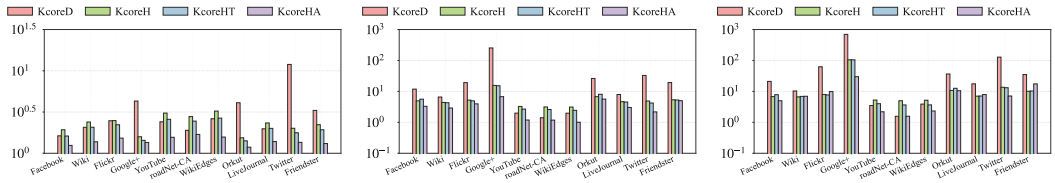


Fig. 9. Accuracy comparison across datasets. Left: mean approximation factor. Middle: MAE. Right: RMSE.

communication cost, averaging about 2.55 uploads per vertex by freezing stabilized vertices early. KcoreHA averages about 3.15 uploads because its adaptive convergence detection occasionally performs additional refinement rounds. Overall, KcoreHT and KcoreHA reduce communication by approximately one order of magnitude relative to KcoreD, with the largest absolute savings appearing on large graphs.

8.3 Effectiveness Evaluation

Evaluation metrics. We evaluate estimation accuracy using approximation factor, mean absolute error (MAE) and root mean squared error (RMSE). Given true coreness c_u^* and estimate $\hat{c}_u$ for each vertex $u \in V$:

$$\text{MAE} = \frac{1}{|V|}\sum_u |\hat{c}_u - c_u^*|, \qquad \text{RMSE} = \sqrt{\frac{1}{|V|}\sum_u (\hat{c}_u - c_u^*)^2}.$$

MAE reflects average deviation, whereas RMSE emphasizes large errors.

The per-vertex approximation factor is

$$\alpha_u = \max\left(\frac{\hat{c}_u}{c_u^*+\epsilon}, \frac{c_u^*}{\hat{c}_u+\epsilon}\right),$$

with $\epsilon > 0$ to avoid division by zero. We report the mean $\overline{\alpha}$ to summarize global accuracy.

Accuracy comparison. Figure 9 shows that across the eleven real-world graphs in our evaluation, KcoreHA consistently outperforms the distributed baseline, while KcoreH and KcoreHT are competitive on average but do not uniformly outperform it. Averaged over all datasets, KcoreHT reduces MAE by about 6.4× and RMSE by about 5.6×, while KcoreH achieves roughly 6.2× lower MAE and 5.6× lower RMSE. Among all methods, KcoreHA is the most accurate overall, yielding about 10.6× and 10.1× reductions in MAE and RMSE, respectively. The mean approximation factor also drops markedly, from about 3.52× for the baseline to 2.33× for KcoreH, 2.02× for KcoreHT, and only 1.42× for KcoreHA. The gains are especially clear on large graphs: on *Orkut*, KcoreH and KcoreHA reduce MAE by about 3.9× and 4.6×; on *LiveJournal*, the reductions are about 1.7× and 2.6×; and on *Twitter* (1.47×10^9 edges), they further increase to about 6.7× and 15.0×, respectively.

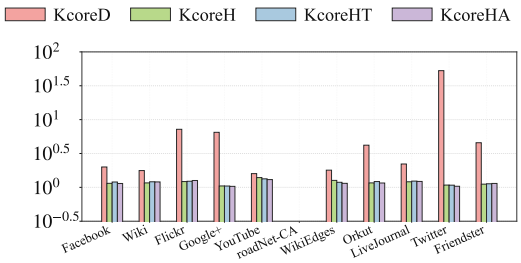


Fig. 10. Mean approximation factor of degree over 50.

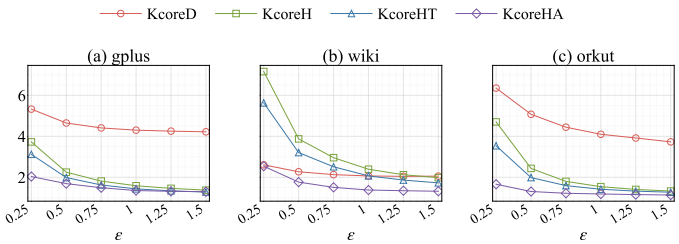


Fig. 11. Sensitivity analysis of privacy budget.

Accuracy for high degree vertices. To further examine performance on structurally important vertices, we restrict attention to high degree ranges and compare the mean approximation factor and error metrics for vertices with deg $\geq$ 100, deg $\geq$ 150, deg $\geq$ 200, and deg $\geq$ 300 across the evaluated graphs. Figure 10 compares the mean approximation factors for nodes with degrees above 50. In this regime, KcoreH is the strongest method overall. Averaged over all datasets with available vertices in each range, its mean approximation factor remains tightly concentrated between 1.15× and 1.20×, while the corresponding MAE increases only moderately from about 12.4 at deg $\geq$ 100 to 23.1 at deg $\geq$ 300. KcoreHT remains competitive on these high degree ranges, but deteriorates modestly in the tail: its mean approximation factor grows from about 1.18× to 1.22×, and its MAE rises from about 15.6 to 25.6. The distributed baseline is substantially less accurate throughout, with average mean approximation factor increasing from 11.5× at deg $\geq$ 100 to 19.3× at deg $\geq$ 300, and MAE increasing from 128.9 to 217.0.

Compared with KcoreH and KcoreHT, KcoreHA has weaker average accuracy on these high-degree subsets, with mean approximation factor roughly between 1.21× and 1.35× and MAE increasing from about 16.5 to 34.0 as the degree threshold becomes more stringent. However, KcoreHA is particularly competitive on the largest heavy tailed graphs. For example, on *Google+*, its MAE is substantially smaller than that of KcoreH for high degree vertices, such as 14.7 versus 39.1 at deg $\geq$ 150. On *Twitter*, it achieves the smallest errors among all methods across the high degree ranges, such as 7.1 compared with 11.8 for KcoreH and 13.5 for KcoreHT at deg $\geq$ 150. Overall, these results indicate that high degree vertices remain the most challenging regime. Iterative H-index refinement remains the most robust strategy in this setting, KcoreHT preserves most of this advantage while reducing unnecessary updates, and KcoreHA offers a stronger adaptive alternative that can be especially effective on very large heavy tailed graphs.

Effect of privacy budget ε. Figure 11 compares the approximation factors of our KcoreHA algorithm and the distributed baseline under varying privacy budgets ε. Across all datasets, larger

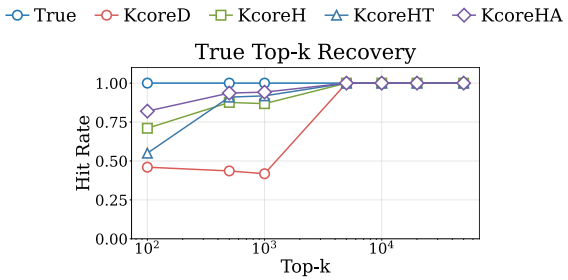


Fig. 12. Google+ circles case study.

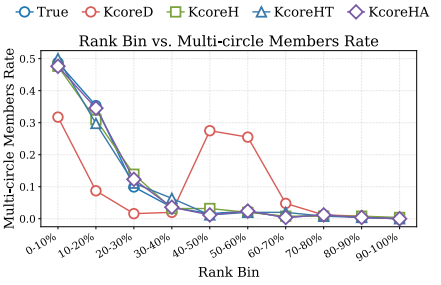


Fig. 13. Rank bin distribution of multi circle users.

ε values lead to lower approximation factors for both methods, as expected under weaker privacy constraints. However, the rate of improvement differs significantly. For large or dense graphs such as *gplus* and *orkut*, KcoreHA already achieves strong accuracy even under tight privacy budgets ($\varepsilon < 0.5$), and maintains clear superiority as ε increases. On the midsize graph *wiki*, the baseline improves steadily as ε grows, but KcoreHA remains consistently better throughout the whole range. Overall, our approach exhibits a more pronounced improvement with increasing ε, achieving much higher accuracy than the baseline in both moderate and high privacy regimes. This demonstrates that KcoreHA can efficiently leverage additional budget to refine estimates and achieves substantial gains when privacy constraints are relaxed.

Distributed robustness. We emulated distributed execution with eight edge-balanced workers. Workers delivered messages at different times, and round-tagged updates were committed after each logical round. We tested ideal execution, 10 ms RTT with 1 Gbps bandwidth, 0–20 ms completion skew, and one worker interruption with checkpoint recovery. Recovery retransmitted cached private reports without additional privacy loss. Across 20 seeds on Facebook, Wiki, and DBLP, all 240 executions exactly matched their local counterparts: $\max_u |\hat{c}_u^{\text{dist}} - \hat{c}_u^{\text{local}}| = 0$, so all accuracy metrics remained unchanged. This agreement is largely by construction because recovery replays cached reports rather than resampling noise. The experiment validates the implementation under message reordering, delay, and checkpoint recovery, but not genuine asynchronous execution or real failures. In this emulation, these effects changed completion time but not privacy or accuracy.

Case study on Google+ circles. We study a Google+ network with ground truth circles to test whether LDP coreness estimation can recover structurally important users. Users appearing in at least two circles are treated as multi-circle members, serving as a proxy for overlap users that connect communities. Figure 12 evaluates ranking quality through the overlap between the true top-k coreness users and the top-k users returned by each method. KcoreHA performs best: its top-100 recovery is about 0.82 versus 0.46 for KcoreD. Moreover, its Spearman correlation is about 0.985 versus 0.200. This indicates that KcoreHA preserves the global coreness ordering much more accurately. Figure 13 examines where multi-circle members appear in each predicted ranking. We split each ranking into ten equal-sized bins and compute the fraction of multi-circle members in each bin. Under the true ranking, the first two bins have rates about 48.8% and 35.3%, both well above the global average of about 10.4%. KcoreHA matches this pattern closely, and KcoreH and KcoreHT still show a clear decline from early bins to later ones. KcoreD is less consistent, with weaker concentration near the front and irregular spikes in the middle bins.

Overall, high coreness users in this network are much more likely to belong to multiple circles. KcoreHA preserves this signal most faithfully under LDP, making it useful for finding community core and cross community bridge users.

9 Conclusion

In this work, we adopt the local edge differential privacy (LEDP) model, allowing each node to protect its incident edges before any information reaches the curator. Leveraging the neighborhood H index operator, we design three integer native LEDP algorithms for coreness estimation: KcoreH, an iterative H index scheme with explicit high probability guarantee; KcoreHT, a transition based variant that reduces unnecessary noisy updates by locking vertices whose estimates have already stabilized; and KcoreHA, an adaptive method that automatically adjusts the number of rounds, allocates privacy budget across rounds, freezes vertices that have already converged, and detects whether the whole graph has converged online. Across a range of real world graphs, these methods achieve up to one to two orders of magnitude speedups over the current best performing work and reduce mean approximation factors by up to 2×, supporting private structural analysis of large scale social, financial, and communication networks.

Acknowledgments

Fan Zhang is partially supported by the National Natural Science Foundation of China under Grant No. 62572140 and the Guangdong Basic and Applied Basic Research Foundation under Grant No. 2024A1515011501.

References

[1] Jun Ai, Yayun Liu, Zhan Su, Fengyu Zhao, and Dunlu Peng. 2021. K-core decomposition in recommender systems improves accuracy of rating prediction. *International Journal of Modern Physics C* 32, 07 (2021), 2150087.

[2] J. Ignacio Alvarez-Hamelin, Luca Dall'Asta, Alain Barrat, and Alessandro Vespignani. 2005. Large scale networks fingerprinting and visualization using the k-core decomposition. *Advances in Neural Information Processing Systems (NeurIPS)* 18 (2005).

[3] Victor Balcer and Salil Vadhan. 2017. Differential privacy on finite computers. *arXiv preprint arXiv:1709.05396* (2017).

[4] Vladimir Batagelj and Matjaž Zaversnik. 2003. An O(m) Algorithm for Cores Decomposition of Networks. *arXiv preprint cs/0310049* (2003).

[5] Felipe T. Brito, Victor A. E. Farias, Cheryl Flynn, Subhabrata Majumdar, Javam C. Machado, and Divesh Srivastava. 2023. Global and Local Differentially Private Release of Count-Weighted Graphs. *Proceedings of the ACM on Management of Data* 1, 2 (2023), 1–25.

[6] Mark Bun and Thomas Steinke. 2016. Concentrated differential privacy: Simplifications, extensions, and lower bounds. In *Theory of cryptography conference*. Springer, 635–658.

[7] Clément L. Canonne, Gautam Kamath, and Thomas Steinke. 2020. The Discrete Gaussian for Differential Privacy. In *NeurIPS*. https://arxiv.org/pdf/2004.00010

[8] Duanbing Chen, Linyuan Lü, Ming-Sheng Shang, Yi-Cheng Zhang, and Tao Zhou. 2012. Identifying influential nodes in complex networks. *Physica A: Statistical Mechanics and its Applications* 391, 4 (2012), 1777–1787. doi:10.1016/j.physa.2011.09.017

[9] Martino Ciaperoni, Edoardo Galimberti, Francesco Bonchi, Ciro Cattuto, Francesco Gullo, and Alain Barrat. 2020. Relevance of temporal cores for epidemic spread in temporal networks. *Scientific reports* 10, 1 (2020), 12529.

[10] Laxman Dhulipala, Quanquan C Liu, Sofya Raskhodnikova, Jessica Shi, Julian Shun, and Shangdi Yu. 2022. Differential privacy from locally adjustable graph algorithms: k-core decomposition, low out-degree ordering, and densest subgraphs. In *2022 IEEE 63rd Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE, 754–765.

[11] Laxman Dhulipala, Murtuza Mittal, and Jeffrey Ullman. 2022. Differentially Private k-core Decomposition: Improved Algorithms for Edge LDP. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[12] Laxman Dhulipala, Murtuza Mittal, and Jeffrey Ullman. 2023. Near-Optimal Differentially Private k-core Decomposition. In *Proceedings of the ACM Symposium on Theory of Computing (STOC)*. 1473–1484.

[13] Xiaofeng Ding, Shujun Sheng, Huajian Zhou, Xiaodong Zhang, Zhifeng Bao, Pan Zhou, and Hai Jin. 2021. Differentially Private Triangle Counting in Large Graphs. *IEEE TKDE* 34, 11 (2021), 5278–5292.

[14] Michael Dinitz, Satyen Kale, Silvio Lattanzi, and Sergei Vassilvitskii. 2024. Almost Tight Bounds for Differentially Private Densest Subgraph. *arXiv preprint arXiv:2308.10316* (2024).

[15] Cynthia Dwork. 2006. *Differential Privacy*. Automata, Languages and Programming, Springer. 1–12 pages.

[16] Cynthia Dwork and Jing Lei. 2009. Differential privacy and robust statistics. In *Proceedings of the forty-first annual ACM symposium on Theory of computing*. 371–380.

[17] Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam Smith. 2006. Calibrating noise to sensitivity in private data analysis. In *Theory of cryptography conference*. Springer, 265–284.

[18] Cynthia Dwork, Moni Naor, Toniann Pitassi, and Guy N Rothblum. 2010. Differential privacy under continual observation. In *Proceedings of the forty-second ACM symposium on Theory of computing*. 715–724.

[19] Cynthia Dwork, Guy N Rothblum, and Salil Vadhan. 2010. Boosting and differential privacy. In *2010 IEEE 51st annual symposium on foundations of computer science*. IEEE, 51–60.

[20] Talya Eden, Quanquan C. Liu, Sofya Raskhodnikova, and Adam Smith. 2023. Triangle Counting with Local Edge Differential Privacy. In *Proceedings of the 50th International Colloquium on Automata, Languages, and Programming (ICALP)*.

[21] Alexandre Evfimievski, Johannes Gehrke, and Ramakrishnan Srikant. 2003. Limiting privacy breaches in privacy preserving data mining. In *Proceedings of the twenty-second ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*. 211–222.

[22] Alireza Farhadi, MohammadTaghi Hajiaghayi, and Elaine Shi. 2021. Differentially Private Densest Subgraph. *arXiv preprint arXiv:2106.00508* (2021).

[23] Arpita Ghosh, Tim Roughgarden, and Mukund Sundararajan. 2012. Universally Utility-Maximizing Privacy Mechanisms. *SIAM J. Comput.* 41, 6 (2012), 1673–1693. doi:10.1137/09076828X

[24] Christos Giatsidis, Fragkiskos D. Malliaros, Nikolaos Tziortziotis, Charanpal Dhanjal, Emmanouil Kiagias, Dimitrios M. Thilikos, and Michalis Vazirgiannis. 2016. A k-core decomposition framework for graph clustering. *arXiv preprint arXiv:1607.02096* (2016).

[25] Yizhang He, Kai Wang, Wenjie Zhang, Xuemin Lin, Wei Ni, and Ying Zhang. 2024. Butterfly Counting over Bipartite Graphs with Local Differential Privacy. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 2351–2364. doi:10.1109/ICDE60146.2024.00186

[26] Yizhang He, Kai Wang, Wenjie Zhang, Xuemin Lin, and Ying Zhang. 2024. Common Neighborhood Estimation over Bipartite Graphs under Local Differential Privacy. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–26. doi:10.1145/3698803

[27] Yizhang He, Kai Wang, Wenjie Zhang, Xuemin Lin, Ying Zhang, and Wei Ni. 2025. Robust Privacy-Preserving Triangle Counting under Edge Local Differential Privacy. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–26. doi:10.1145/3725348

[28] Quentin Hillebrand, Vorapong Suppakitpaisarn, and Tetsuo Shibuya. 2023. Communication Cost Reduction for Subgraph Counting under Local Differential Privacy via Hash Functions. *arXiv preprint arXiv:2312.07055* (2023).

[29] Jacob Imola, Takao Murakami, and Kamalika Chaudhuri. 2021. Communication-Efficient Triangle Counting under Local Differential Privacy. *arXiv preprint arXiv:2110.06485* (2021).

[30] Jacob Imola, Takao Murakami, and Kamalika Chaudhuri. 2021. Locally differentially private analysis of graph statistics. In *30th USENIX security symposium (USENIX Security 21)*. 983–1000.

[31] Jacob Imola, Takao Murakami, and Kamalika Chaudhuri. 2022. {Communication-Efficient} triangle counting under local differential privacy. In *31st USENIX security symposium (USENIX Security 22)*. 537–554.

[32] Vishesh Karwa, Sofya Raskhodnikova, Adam Smith, and Grigory Yaroslavtsev. 2011. Private Analysis of Graph Structure. *PVLDB* 4, 11 (2011), 1146–1157.

[33] Vishesh Karwa and Salil Vadhan. 2017. Finite sample differentially private confidence intervals. *arXiv preprint arXiv:1711.03908* (2017).

[34] Shiva Prasad Kasiviswanathan, Homin K Lee, Kobbi Nissim, Sofya Raskhodnikova, and Adam Smith. 2011. What can we learn privately? *SIAM J. Comput.* 40, 3 (2011), 793–826.

[35] Maksim Kitsak, Lazaros K. Gallos, Shlomo Havlin, Fredrik Liljeros, Lev Muchnik, H. Eugene Stanley, and Hernán A. Makse. 2010. Identification of influential spreaders in complex networks. *Nature Physics* 6, 11 (2010), 888–893. doi:10.1038/nphys1746

[36] Rong-Hua Li, Lu Qin, Jeffrey Xu Yu, and Rui Liu. 2014. Dynamic Maintenance of k-core Decomposition in Large Graphs. *Proceedings of the ACM SIGMOD* (2014), 1109–1120.

[37] Linyuan Lü, Tao Zhou, Qian-Ming Zhang, and H. Eugene Stanley. 2016. The H-index of a network node and its relation to degree and coreness. *Nature Communications* 7 (2016), 10168. doi:10.1038/ncomms10168

[38] David W. Matula and Leland L. Beck. 1983. Smallest-last ordering and clustering and graph coloring algorithms. *J. ACM* 30, 3 (1983), 417–427.

[39] Gang Mei, Jingzhi Tu, Lei Xiao, and Francesco Piccialli. 2021. An efficient graph clustering algorithm by exploiting k-core decomposition and motifs. *Computers & Electrical Engineering* 96 (2021), 107564.
[40] Higor S. Monteiro, Shaojun Luo, Saulo D. S. Reis, Carles Igual, Antonio S. Lima Neto, Matias Travizan, José Soares de Andrade Jr, and Hernán A. Makse. 2021. Superspreading k-cores at the center of COVID-19 pandemic persistence. *Bulletin of the American Physical Society* 66 (2021).
[41] Pranay Mundra, Charalampos Papamanthou, Julian Shun, and Quanquan C. Liu. 2025. Practical and Accurate Local Edge Differentially Private Graph Algorithms. *Proceedings of the VLDB Endowment* 18, 11 (2025), 4199–4213. doi:10.14778/3749646.3749687
[42] Kobbi Nissim, Sofya Raskhodnikova, and Adam Smith. 2007. Smooth Sensitivity and Sampling in Private Data Analysis. In *STOC*. 75–84.
[43] Zhan Qin, Ting Yu, Yin Yang, Issa Khalil, Xiaokui Xiao, and Kui Ren. 2017. Generating synthetic decentralized social graphs with local differential privacy. In *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*. 425–438.
[44] Ahmet Erdem Sariyüce, Bugra Gedik, Shantanu R. Sharma, and Srikanta Tirthapura. 2016. Streaming Algorithms for k-core Decomposition. *Proceedings of the VLDB Endowment* 10, 3 (2016), 209–220.
[45] Stephen B. Seidman. 1983. Network structure and minimum degree. *Social Networks* 5, 3 (1983), 269–287.
[46] Matteo Serafino, Higor S Monteiro, Shaojun Luo, Saulo DS Reis, Carles Igual, Antonio S Lima Neto, Matías Travizano, José S Andrade Jr, and Hernán A Makse. 2022. Digital contact tracing and network theory to stop the spread of COVID-19 using big-data on human mobility geolocalization. *PLOS Computational Biology* 18, 4 (2022), e1009865.
[47] Elaine Shi, HTH Chan, Eleanor Rieffel, Richard Chow, and Dawn Song. 2011. Privacy-preserving aggregation of time-series data. In *Annual Network & Distributed System Security Symposium (NDSS)*. Internet Society.
[48] Kijung Shin, Tina Eliassi-Rad, and Christos Faloutsos. 2016. CoreScope: Graph Mining Using k-Core Analysis – Patterns, Anomalies and Algorithms. In *Proceedings of the IEEE International Conference on Data Mining (ICDM)*. IEEE, 469–478. doi:10.1109/ICDM.2016.0055
[49] Qingqing Ye, Haibo Hu, Man Ho Au, Xiaofeng Meng, and Xiaokui Xiao. 2020. LF-GDPR: A Framework for Estimating Graph Metrics with Local Differential Privacy. *IEEE TKDE* 34, 10 (2020), 4905–4920.
[50] Wei Zeng, An Zeng, Hao Liu, Ming-Sheng Shang, and Tao Zhou. 2014. Uncovering the information core in recommender systems. *Scientific Reports* 4 (2014), 6140.
[51] Yuting Zhang, Wei Ni, Kai Wang, and Conggai Li. 2025. Truss Decomposition Under Edge Local Differential Privacy. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE, 2670–2683. doi:10.1109/ICDE65448.2025.00201

Received 17 April 2026; revised 19 August 2026; accepted 12 September 2026